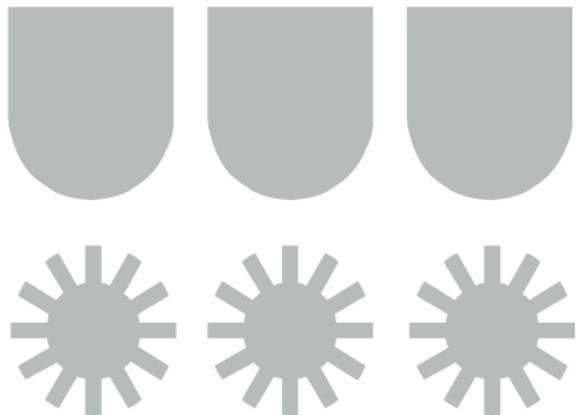




UNIVERSIDAD DEL BÍO-BÍO

Introducción a la Programación

Introducción a Python



Introducción a la programación

- Curso para aprender las nociones básicas para desarrollar programas que resuelven problemas simples
- Objetivos clases práctica y laboratorio:
 1. Práctica/Ejercitar conocimientos de clases teóricas (**aprender a programar**)
 2. **Desarrollar habilidades** para elaborar algoritmos y programas



Pregunta 1

Si son nuevos en un trabajo y les pide hacer algo que no saben

¿Qué hacen?

Respuesta:

1. **Preguntan** (más rápido preguntar a quien sabe)
2. **Investigan** (si no hay nadie a quien preguntar, busco soluciones o recurso de aprendizajes que sirvan)
3. **Se las ingenian** (si 1 y 2 no resultan, usar habilidades de resolución de problemas)



Pregunta 2

Si en clases de laboratorio de intro a la prog les piden hacer algo que no sabe

¿Qué hacen?

Respuesta:

1. **Se las ingenian** (desarrollo de habilidades cognitivas para resolver problemas)
2. **Investigan** (buscar si se han resuelto ejercicios similares en clases o estudiando)
3. **Preguntan** (si 1 y 2 no dan resultado, preguntar a “quien sepa”)

Respuesta: “sujeta a factor tiempo”



¿Qué es Python?

- Lenguaje de programación de **alto nivel** (más cercano a lenguaje humano) de **código abierto**.
- Uno de los lenguajes de programación más usados en la actualidad utilizado en el **desarrollo de aplicaciones, análisis de datos, inteligencia artificial**, entre otros.
- Fácil de aprender su sintaxis
- Es un **lenguaje Interpretado**, esto significa que se ejecuta directamente línea por línea.



Variables y tipos de datos

- **Variable:** se utilizan para almacenar valores de datos en Python una variable se crea en el momento que se le asigna un valor (la variable toma el tipo de dato del valor que se le asigna)

Tipos de datos:

- **Números:** enteros, flotantes (reales) y complejos
- **Cadenas (string):** Texto delimitado por comillas
- **Booleanos:** valores **true** o **false**

Casting: Es posible convertir un tipo de dato en otro explícitamente

- **int()**
- **float()**
- **str()**
- **Ingreso de datos:** función **input()**
- **Mostrar por pantalla (consola):** función **print()**



Bifurcaciones

- Una bifurcación es una estructura de control que utiliza una sentencia lógica que se utiliza para tomar decisiones en el código basadas en condiciones.

Estructuras:

- **if:** Evalúa una condición inicial
- **elif:** (abreviación de **else if**) Evalúa una nueva condición si la anterior fue falsa
- **else:** Captura cualquier caso que no haya cumplido las condiciones previas



Ejercicios 1

- a) Desarrolle un programa en Python que permita ingresar la edad de una persona y luego muestre si la persona es menor o mayor de edad (solo utilizando bifurcaciones)
- b) Desarrolle un programa en Python que permita ingresar la edad de una persona y luego muestre si la persona es menor de edad, adulto sin ser de tercera edad o una persona adulto mayor



Iteraciones

Estructuras de control que permiten ejecutar un conjunto de líneas de código mientras se cumpla una condición

Estructuras:

- **while:** ejecuta las instrucciones mientras una condición sea verdadera
- **for:** Esencial para ejecutar un bloque de código un número específico de veces; genera una secuencia de números que, por defecto, comienza en 0



Iteraciones: while

Ejemplos while:

a)

```
1  i = 1
2  while i < 7:
3      print(i)
4      i += 1
```

b)

```
1  i = 1
2  var = 0
3  while i <= 7:
4      var = var + i
5      i += 1
6
7  print(var)
```

Describe que hace cada ejemplo



Iteraciones: for

Ejemplos for

a)

```
1  for x in range(7):  
2  |    print(x)
```

b)

```
1  for x in range(1,7+1):  
2  |    print(x)
```

c)

```
1  for x in range(1,10+1,2):  
2  |    print(x)
```

Describe que hace cada ejemplo

d)

```
1  estudiantes = ["Juan", "Camila", "Oliver"]  
2  
3  for x in estudiantes:  
4  |    print(x)
```



Funciones

Definición: Un bloque de código que solo se ejecuta cuando es llamado

Modularidad: Ayudan a organizar el código en partes reutilizables

Componentes:

Argumentos: Información que se pasa a la función (incluyendo *args para un número indefinido de argumentos)

Retorno: Las funciones pueden devolver datos como resultado



Funciones

Ejemplos funciones:

a)

```
1 def funcion_hola_mundo():  
2     print("hola mundo")
```

b)

```
1 def funcion_sumar(a,b):  
2     return a+b
```

Describa que hace cada ejemplo



Módulos

¿Qué son?: Archivos que contienen un conjunto de funciones que se pueden incluir en una aplicación.

Módulos integrados: Python incluye librerías listas para usar como **math** para operaciones matemáticas complejas, **random** para generar números aleatorios y **statistics**



Ejercicio 2

Desarrolle un programa en Python que permita desarrollar las siguientes operaciones:

1. Sumar todos los valores enteros positivos desde 1 a n
2. Obtener el factorial de N
3. Ingresar los nombres de N estudiantes
4. Mostrar los nombres de los N estudiantes ingresados
5. Ingresar las edades de N estudiantes universitarios
6. Obtener el promedio de los n estudiantes universitarios

